

Document Number: P0975R0

Date: 2018-03-10

Audience: Evolution, SG1, SG14, LEWG, LWG

Revises: none

Reply to: Gor Nishanov (gorn@microsoft.com)

Impact of coroutines on current and upcoming library facilities

- [1. Overview](#)
- [2. Standard coroutine types](#)
- [3. Existing library facilities](#)
- [4. Incoming papers with generator interactions](#)
 - [4.1. Ranges](#)
- [5. Brief tutorial on awaitables from a library perspective](#)
- [6. Incoming papers with async interactions](#)
 - [6.1. Executors](#)
 - [6.2. Concurrency TS](#)
 - [6.2.1. future](#)
 - [6.2.2. when_all](#)
 - [6.2.3. when_any](#)
 - [6.3. Other future-like types](#)
 - [6.4. Parallelism TS](#)
 - [6.4.1. Task Blocks](#)
 - [6.5. std::expected](#)
 - [6.6. Unified Call Syntax](#)
 - [6.7. Revised Latches and Barriers for C++20](#)
 - [6.8. Parallel algorithms](#)
 - [6.9. Concurrent Queues](#)
 - [6.10. RCU & Hazard pointers](#)
 - [6.11. Networking Technical Specification](#)
 - [6.11.1. async API: simple form](#)
 - [6.11.2. async API: adding an executor](#)
 - [6.11.3. async API: altering default allocator](#)
- [7. Conclusion](#)
- [8. Acknowledgement](#)
- [9. References](#)
- [10. Appendix](#)

- [10.1. Full listing of hand-crafted state machine matching the example in "Networking Technical Specification" section of the paper.](#)

1. Overview

A coroutine is a generalization of a function that in addition to usual control flow operations such as call and return, can suspend execution of itself and yield control back to the caller with an ability to resume execution at a later time. In C++, coroutines were explicitly designed to efficiently and succinctly support the following use patterns:

- asynchronous tasks, where `co_await <expr>` suspends a coroutine while waiting for the result of an expression) and the coroutine is resumed once the result is available
- generators, where `co_yield <expr>` suspends the coroutine yielding the result of the expression to the consumer and the coroutine is resumed once the consumer asks for the next value
- asynchronous streams, which can be thought as an asynchronous version of a generator where both `co_await` and `co_yield` can be used.

Unlike most other languages that support coroutines, C++ coroutines are open and not tied to any particular runtime or generator type and allow libraries to imbue coroutines with meaning, whereas the compiler is responsible solely for efficient transformation of a function to a state machine that is the foundation of the coroutine.

While openness of coroutines allows easy integration into existing concurrency runtimes, we do not yet have great out of the box coroutine experience for C++20 as we do not have standard coroutine types, like task or generator and we do not have coroutine bindings with any upcoming concurrency and networking facilities in papers in flight.

This document describes what is missing, what is the impact of coroutines on existing library facilities (almost none) and what is the impact of coroutines on papers in progress (lots).

2. Standard coroutine types

There are two proposals en route for Rapperswil that will bring first two coroutine types to the library: generators and zero-overhead tasks. There will be also a paper on a zero-overhead adapters to consume Networking TS *_async APIs.

This is the minimal level of library support that allows C++20 to have great out of the box experience.

For async code, users should be able to write efficient and pretty async networking code:

```
task<> session(io_context &io, ip::tcp::socket s, size_t block_size) {
    std::vector<char> buf(block_size);
    for (;;) {
        auto n = co_await s.async_read_some( buffer(buf.data(),
                                                    block_size));
        n = co_await async_write(s, buffer(buf.data(), n));
    }
}
```

```
}  
}
```

Users will be able to write lazily generated views with generator coroutines:

```
template <typename T> // note, no upper bound, goes to "infinity"  
generator<T> squares() {  
    for (T i = {}; ; ++i)  
        co_yield i * i;  
}
```

and then use them with ranges and standard library algorithms:

```
int sum = accumulate(squares<int>()  
    | view::transform([](int i){ return i*i; })  
    | view::take(10), 0);
```

With regard to asynchronous streams, it is an area of active research [RxMarbles, RxCpp] and we have no plans to offer any standard coroutine types supporting this use case for C++20.

3. Existing library facilities

There is minimal impact on existing library types. One of the potential types that may interact with coroutines is `std::future`, however, in its current form, it does not have an ability to provide an asynchronous notification when the `future` is completed and therefore cannot be efficiently `co_await`-ed upon. We have several options to consider:

1. If then makes it to `std::future` by C++20, we can make `std::future` completely coroutine aware. We would likely need to give guidance when heavy-weight `std::future` needs to be used with coroutines vs. when to use zero-overhead `task` type.
2. If then does not make it to the `std::future` in C++20. We cannot make `co_await` await on the future, but we can provide bindings allowing coroutines to author `std::future` (as in coroutine returns `std::future` to represent the eventual result produced by the coroutine).
3. We can also decide that `std::future` will not interact with coroutines at all.

The approach chosen for the `std::future` will apply to `std::shared_future` as well.

4. Incoming papers with generator interactions

4.1. Ranges

Zero-overhead generators are unavoidably move-only. However, [Ranges TS](#), at the moment, requires copyability for view composition. We would need to relax copyable requirement, at minimum for input views, or possibly for *all* view categories.

Note that [Range-v3](#) currently implements a “compromise” solution that allows input views to be move-only and requires copy for forward views.

5. Brief tutorial on awaitables from a library perspective

Before we proceed further, it would be valuable to briefly look at the important coroutine related concept, an awaitable.

```
template <typename A>
concept SimpleAwaitable = requires(A a, std::coroutine_handle<> h) {
    { a.await_ready() } -> bool;
    a.await_suspend(h);
    a.await_resume();
};
```

An *await-expression*, i.e. `co_await <expr>` requires `<expr>` to be an awaitable (or to have an overload of operator `co_await` that when applied to `<expr>` will return an awaitable).

Let's look at some simple awaitable examples.

The simplest awaitable possible is `suspend_never` that acts as a noop when awaited upon:

```
struct suspend_never {
    constexpr bool await_ready() const { return true; }
    constexpr void await_suspend(coroutine_handle<>) const {} // never called due
    constexpr void await_resume() const {}
};
```



And here is slightly more complicated awaitable that suspends and enqueues a coroutine for later resumption if some resource is not available:

```
struct get_resource {
    resource_manager& rm;
    optional<resource_manager::value_type> result;
    constexpr bool await_ready() const {
        result = rm.try_get_resource();
        return result.has_value();
    }
    void await_suspend(coroutine_handle<> h) {
        rm.enqueue(h, result);
    }
};
```

```

    }
    auto await_resume() { return move(result.value()); }
};

```

With this awaitable we can write `auto r = co_await get_resource{rm};`. If a resource is available, `await_ready` returns `true`, and proceeds to unpack the result in `await_resume`, otherwise, the coroutine is suspended and enqueued into a queue of coroutines waiting for a resource. When resource becomes available, `resource_manager`, will pick a coroutine from the queue, update the optional and resume the coroutine.

An important property of how *await-expression* is specified by Coroutines TS is that once `await_ready()` is called on awaitable, related calls to `await_suspend()` and `await_resume()` will be called on the same awaitable object. This allows to use awaitable as a convenient store for per operation context required by an operation that *awaitable* represents. If a library performs transformations on an *awaitable* and directly invokes `await_xxx` functions, it should preserve aforementioned property.

Hopefully this section serves as a useful reminder of what coroutine awaitables are.

6. Incoming papers with async interactions

Overall, we need to evaluate papers that provide asynchronous interfaces via callbacks or by returning future-like objects and provide awaitable interfaces. We also need to evaluate papers that offer blocking APIs, where blocking is performed by the library facility and decide whether a non-blocking awaitable version of the API is desired.

Ideally, we would use CompletionToken transformations of Networking TS that can automatically convert from callback based APIs into awaitable or future based APIs. However, at the moment, CompletionToken transformations are not zero-overhead and while this might get resolved before C++20, this paper describes plan B, where we offer APIs with hand-crafted awaitables.

We also need to develop guidance for recommended naming for asynchronous/awaitable vs blocking functions

At the moment we have somewhat inconsistent naming coming from different papers. For example:

- `non_blocking_push` vs `push` (blocking) [Concurrent Queues]
- `async_read` vs `read` (blocking) [Networking TS]
- `arrive` (non-blocking) vs `arrive_and_wait` (blocking) [Revised Latches and Barriers]
- `synchronize_rcu` and `rcu_barrier` (both blocking). How should we call non blocking versions? [RCU & Hazard]

Another aspect to consider, how to express that the user prefers to get an error reported as `expected<T, error_code>` as opposed to a throw. It can take several possible shapes:

```

co_await s.async_read(buf); // will throw on failure or return T

co_await s.async_read(buf).as_expected(); // await will return expected<T,error_
co_await as_expected(s.async_read(buf)); // same, without UFC

```

```
co_await s.async_read(buf, ec); // will return some default value and error will
```

With library facilities, sometimes needing to abandon the callbacks they accepted in the APIs due to shutdown or other considerations, we need to decide a policy of how that will be expressed:

1. awaitable are completed with `errc::cancelled` or cancelled exception
2. awaitable are abandoned and destructor of the awaitable is responsible for cancelling the coroutine.
3. mixture of both approaches (as it is the case in Networking TS today)

And finally, given that we may be adding a lot of awaitables to the library APIs, we need to find an appropriate consistent wording to describe them and provide it as guidance to paper authors.

Note that not all blocking APIs require an awaitable. If the underlying facility is not providing completion notification, burning a thread to fulfill an awaitable requirements is probably an overkill.

Do not do the following!!!

```
// desired usage:
task<> DoSomething(mutex& m) {
    unique_lock<mutex> lock = co_await lock_or_suspend{m};
    // ...
}

// implementation
struct lock_or_suspend {
    unique_lock<mutex> lock;
    lock_or_suspend(mutex & mut)
        : lock(mut, try_to_lock) {}

    bool await_ready() const { return lock.owns_lock(); }
    void await_suspend(coroutine_handle<> h) {
        std::thread t([this, h]{ lock.lock(); h(); });
        t.detach();
    }
    unique_lock<mutex> await_resume() { return move(lock); }
};
```

6.1. Executors

Executors expose six flavors of APIs that take callbacks and initiate execution:

Name	Cardinality	Directionality
execute	single	oneway

Name	Cardinality	Directionality
twoway_execute	single	twoway
then_execute	single	then
bulk_execute	bulk	oneway
bulk_twoway_execute	bulk	twoway
bulk_then_execute	bulk	then

We will focus on the first three. Whatever the decision is made about non-bulk execution functions will be applied to `bulk_` versions.

API	return type	meaning
<code>x.execute(f)</code>	<code>void</code>	execute <code>f</code> on executor <code>x</code>
<code>x.twoway_execute(f)</code>	<code>Future<R></code>	execute <code>f</code> on executor <code>x</code> and return result as <code>Future</code>
<code>x.then_execute(f, fut)</code>	<code>Future<R></code>	execute <code>f</code> on executor <code>x</code> after <code>Future fut</code> completes

During executor design discussions the need for splitting the execution functions into void returning `OneWay` and `Future` returning `TwoWay` was questioned:

Question	Answer
Why not <code>TwoWay</code> always?	one-way execution functions avoid the cost of a future object
Why not <code>OneWay</code> always?	Two-way execution functions allow executors to participate directly in synchronization rather than require inefficient synchronization out-of-band.

With coroutines, the distinction between `OneWay` executors and `TwoWay` is not necessarily relevant. Unless the underlying API that executor abstracts away always have to return a heavy-weight future, two-way APIs can simply return an awaitable which is a simple struct with possibly a single pointer/reference member referring to the executor. `TwoWay` API returning an awaitable is as efficient as a `OneWay` therefore they can be collapsed into one.

The proposed mapping between `OneWay` executor APIs and their awaitable counterparts could look like this:

API	expressed as	meaning
<code>x.execute(f)</code>	<code>co_await execute_on(x)</code>	resume coroutine on executor <code>x</code>
<code>x.twoway_execute(f)</code>	<code>co_await execute_on(x)</code>	resume coroutine on executor <code>x</code> (same as <code>OneWay</code>)
<code>x.then_execute(f, fut)</code>	<code>co_await execute_on(x, fut)</code>	resume coroutine on executor <code>x</code> after awaitable <code>fut</code> completes

There could be alternative spellings for ThenExecute if SG1/LEWG so desires. Here are a few suggestions (some are not necessarily good):

```
co_await execute_on(x, async_read()); // basic form
co_await async_read().execute_on(x);  // UFC flavored form
co_await async_read().via(x);         // SemiFuture inspired form
co_await x << async_read();           // iostream inspired form
```

Also, we encourage authors of Executors paper to explore whether futures in executor APIs can be replaced with awaitables.

Here is a complete implementation of on_execute(x):

```
template <OneWayExecutor E>
auto execute_on(E e) {
    struct Awaitable {
        E e;
        constexpr bool await_ready() const { return false; }
        void await_suspend(coroutine_handle<> h) { e.execute(h); }
        constexpr void await_resume() const {}
    }
    return Awaitable{e};
}
```

and one possible implementation of on_execute(x, a):

```
template <OneWayExecutor E, Awaitable A>
auto execute_on(E e, A&& a) { // deduces to task<result-of-awaiting-on-a>
    auto result = co_await forward<A>(a);
    co_await on_execute(e);
    co_return result;
}
```

6.2. Concurrency TS

Concurrency TS [P0159] covers three areas:

1. atomic_shared_ptr and friends (voted in C++20, no coroutine interaction)
2. latches and barriers (covered in later discussion about p0666r1)
3. std::future improvement

In this section we will focus on the third part and throughout this section future and shared_future will be referring to std::experimental::future and std::experimental::shared_future types of the TS and not to std::future/std::shared_future of the C++ standard.

6.2.1. future

While the interface of the `future` described in the Concurrency TS allows graceful interactions with coroutines, we need to evaluate what are the use cases for the `future` that are not already covered by coroutines using zero-overhead `task` type and whether having two coroutine types with pretty names would create confusion among the users.

The approach chosen for the `future` will apply to `shared_future` as well.

6.2.2. when_all

Concurrency TS offers two `when_all` functions that convert a Sequence of `futures` or `shared_futures` into a `future` of a sequence of completed `futures`.

It can handle futures with heterogenous result type:

```
future<void> x;
shared_future<int> y;
// when_all(x,y) => future<tuple<future<void>, shared_future<int>>>
```

and with homogeneous result types:

```
std::list<future<int>> list;
// when_all(list.begin(), list.end()) => future<vector<future<int>>>
```

We recommend that `when_all` to be modified to accept a sequence of arbitrary awaitables and return an awaitable of result type `tuple` or some container type populated with `expected<T, exception_ptr>` that hold the result of awaiting on a particular awaitable.

This design allows to push the decision about how to deal with exceptions on to the user.

An alternative design could return an awaitable of sequence of values themselves and exceptions could be reported in a form of an `exception_list`. SG1 is actively exploring what would be the best way to deal with exceptions which occur concurrently [[P0797R1](#)].

The algorithm `when_all` is useful when composing awaitable types with standard library algorithms or ranges. For example:

```
int x = accumulate( co_await when_all(x.async_read(), y.async_read())
                   | reduce_exceptions(), 0);
```

where `reduce_exceptions` view adapter will convert `expected<T, E>`s into `T`s applying appropriate exception reduction policy if some `expected`s store failures.

6.2.3. when_any

Concurrency TS also offers `when_any` algorithm that is similar to `when_all`, with the exception that it will complete its future as soon as at least one of the input futures have completed.

Note, that `when_any` behavior is sub-optimal today due to absence of cancellation.

```
std::list<future<int>> list;
list.push(std::async([]{})); // will block in destructor
list.push(async_read()); // will not block in destructor
// when_any(list.begin(), list.end()) => future<pair<size_t, vector<future<int>>>
// when_any in this case will act like when_all
```

However, we foresee that [Networking TS](#) may evolve along the lines suggested in [\[Networking & Threadpools\]](#) and will provide hierarchical lifetime and cancellation domains, called at the moment, `tp_context`. With that infrastructure in place, we can generalize `when_any` and make it behave reasonably in the presence of awaitables and futures with RAII semantics.

`when_any` would have to accept a reference to a cancellation domain `tp_context` and a sequence of arbitrary awaitables. As soon as at least one awaitable have completed (either successfully or with error), `when_any` will initiate cancellation and will wait until **all** awaitables have completed. The return value will return an awaitable of a pair of index and a container of `expected<T, exception_ptr>`. Index will indicate which of the awaitables have completed first. (With respect to decision whether to represent the failures using `expected` or `exception_list`, behavior of `when_any` should match `when_all`).

We can also foresee a desire to have `when_any_successful` or something along these lines where it will await until either all awaitables have completed with failure or at least one completed successfully. With this algorithm we can easily write something like this:

```
tp_context& tp; // cancellation domain
auto t = co_await when_any_successful(tp, tcp::connect(tp, addr1),
                                     tcp::connect(tp, addr2));
// pick whichever connection was successful and use it
```

6.3. Other future-like types

We have several proposals looking into possible evolution of the future:

- Back to the `std2::future` Part II [P0701R2]
- A strawman Future API [P0904R0]
- Towards a Good Future [P0676R0]

Though all of the futures presented in those papers can gracefully interact with coroutines, we would like to understand what scenarios not already covered by coroutines + awaitables they intended for and whether there is a need to have a heavy-weight future type in C++ at all.

6.4. Parallelism TS

6.4.1. Task Blocks

We encourage authors to evaluate whether it is possible to use coroutine to get close to the original cilk vision. For example, a famous cilk fibonacci example:

```
int fib(int n)
{
    if (n < 2)
        return n;
    int x = cilk_spawn fib(n-1); // continuation stealing
    int y = fib(n-2);
    cilk_sync;
    return x + y;
}
```

can be expressed with coroutines as:

```
cilk_task<int> fib(int n)
{
    if (n < 2)
        return n;
    auto x = co_await cilk_spawn(fib(n-1)); // continuation stealing
    auto y = fib(n-2);
    co_await cilk_sync(x,y);
    return x.get() + y.get();
}
```

If spawn/sync is preferable way to express fork/join computations, is there a need for a separate task_block facility?

6.5. std::expected

While it is possible to add coroutine bindings to make coroutines to author or consume objects of `expected<T,E>` type, we are not advocating doing those for C++20.

However, `expected` offers a convenient way to modify awaitables that return values and report errors via exceptions into awaitables that return `expected<T,E>` where error type `E` of is dependent on the operation. If underlying API that awaitable wraps using `error_code` as reporting mechanism, `E` would be `error_code`. Otherwise, it will be an `exception_ptr`.

```
co_await s.async_read(buf); // will throw on failure or return T
co_await s.async_read(buf).as_expected(); // await will return expected<T,error_
```



A bikeshed name for awaitable modifier suggested is `as_expected`.

6.6. Unified Call Syntax

It is impractical for every awaitable to include modifiers, such as `on_executor`, `via` or `as_expected`. Free functions `via`, `on_executor` or `as_expected` can transform arbitrary awaitables to alter their behavior (unless a particular awaitable can be implemented more efficiently with an awaitable specific implementation, in the latter case, the technique similar to `swap` can be used).

However the `.modifier()` syntax is compelling when composing modifiers on awaitable. We have a great hope that some version of Unified Call Syntax [N4474] will be available in C++20 to keep modifier syntax elegant and unobtrusive.

6.7. Revised Latches and Barriers for C++20

The author of "P0666r1: Revised latches and barriers" proposes to extended `latch` and `barrier` classes with non-blocking future based APIs `arrive` for some unspecified future type.

We encourage the author to explore an alternative where `arrive` will be returning an awaitable. The benefit of an awaitable that it is a simple struct returned by value, has no overhead, can be awaited upon and can be trivially converted into any future type that has coroutine binding via a simpler adapter:

```
template <typename T, Awaitable<T> A>
SomeFuture<T> as_some_future(A && a) {
    co_return co_await std::forward<A>(a);
}
```

6.8. Parallel algorithms

Invoking Algorithms Asynchronously paper [p0361r1] correctly argues that `fork/join` parallelism of C++17 parallel algorithms imposes an implicit barrier onto the parallel execution flow that impedes parallel efficiency and efficient resource utilization.

The paper proposes to add a new execution policy to the parallel algorithms that will alter their return value from `void` to some `future_type`.

As with the case of latches and barriers paper we recommend returning an awaitable instead of the future.

6.9. Concurrent Queues

Currently, "P0260r1: Concurrent Queues" paper proposes queues that have blocking and non-blocking APIs for pushing and popping. Non-blocking APIs have form:

```
queue_op_status queue::nonblocking_push(const Element&);
```

If the operation would block, return `queue_op_status::busy`. Otherwise, if the queue is full, return `queue_op_status::full`. Otherwise, push the `Element` onto the queue. Return `queue_op_status::success`.

If authors of P0260r1 decide that awaitable version is desirable, they would need to add:

```
| Awaitable<void> queue::async_push(const Element&);
```

where the return type is some awaitable type that will result in suspension of a coroutine if the queue is full or not available for immediate push. Coroutine should be resumed when the element is enqueued. Eventual result of the awaitable should be of type `void`.

6.10. RCU & Hazard pointers

RCU and hazard pointers paper [p0566r4] exposes two free functions that perform blocking operations:

```
void synchronize_rcu() noexcept;  
void rcu_barrier() noexcept;
```

We would like to ask authors, if it would make sense to provide an awaitable version of those APIs, for example:

```
SomeAwaitable async_synchronize_rcu() noexcept;  
SomeAwaitable async_rcu_barrier() noexcept;
```

that would suspend the coroutine until the it is safe to proceed and resume it afterwards.

6.11. Networking Technical Specification

Networking TS [N4711] is the proposal that would benefit the most from adding coroutine support. Asynchronous programming offered by Networking TS is simplified dramatically when combined with coroutines.

The following short code fragment:

```
task<> session(tcp::socket s, size_t block_size) {  
    std::vector<char> buf(block_size);  
    for (;;) {
```

```

    size_t n = co_await async_read_some(s, buffer(buf.data(), block_size));
    co_await async_write(s, buffer(buf.data(), n));
}
}

task<> server(io_context& io, tcp::endpoint const& endpoint, size_t block_size)
{
    tcp::acceptor acceptor(io, endpoint);
    acceptor.listen();
    for (;;)
        spawn(io, session(co_await async_accept(acceptor), block_size));
}

```

replaces a hand-crafted state machines 100 lines long that you would have to write otherwise (see [Appendix](#) for full listing as it is too long to insert into main body of this paper).

Because providing coroutine bindings is of critical importance to Networking TS, we will offer a dedicated paper on this subject. This section offers a brief overview.

6.11.1. async API: simple form

Here is an example of a typical async API exposed by networking TS:

```

s.async_write(buffer(buf, size),
    [](error_code const& ec, size_t n) { handle_write(ec, n); });

```

Awaitable flavor of the API above would look like:

```

size_t n = co_await s.async_write(buffer(buf, size));

```

Where a `system_error` exception will be thrown if operation completes with error. If immediate error handling is desired, `as_expected()` modifier can be used to alter this behavior:

```

expected<size_t, error_code> result = co_await s.async_write(buffer(buf, size)).
if (!result.has_value()) {
    // deal with the error
}

```

6.11.2. async API: adding an executor

If a user desires to alter the executor on which completion of the asynchronous operation will be invoked, it can use `bind_executor` wrapper around the lambda passed as a last parameter to an async API:

```
s.async_write(buffer(buf, size),
              bind_executor(strand,
                            [] (error_code const& ec, size_t n) { handle_write(ec, n); }));
```

In a coroutine, `via` awaitable modifier can be used to achieve the same purpose:

```
size_t n = co_await s.async_write(buffer(buf, size)).via(strand);
```

6.11.3. async API: altering default allocator

Networking TS allocates a small per operation context for every asynchronous operation. User can supply an allocator that would replace a default allocator used by networking TS implementation. Networking TS does not offer a simple wrapping helper (like `bind_executor`) that allows to pair a callable with a matching allocator, it is simple to write and `make_custom_alloc_handler` in the example below is the described wrapper:

```
s.async_write(buffer(buf, size),
              make_custom_alloc_handler(alloc,
                                        [] (error_code const& ec, size_t n) { handle_write(ec, n); }));
```

When Networking TS is modified to use awaitables, it can utilize the storage available in the awaitable and may not need any dynamic allocations. However if user desires to control where per operation context goes, it can use `using_allocator()` modifier as shown below:

```
size_t n = co_await s.async_write(buffer(buf, size)).using_allocator(alloc);
```

7. Conclusion

Coroutine TS allows library authors to offer users elegant and efficient access to their facilities when used with coroutines. Given the guidance that big language features with broad library impact need to land early, we recommend to merge Coroutines TS into the working paper as soon as possible to give library authors a stable foundation to work on.

8. Acknowledgement

Great thanks to Casey Carter for review and suggestions.

9. References

[Executors]: [P0761R2: Executors Design Document](#)

[NetworkingTS]: [N4711: C++ Extensions for Networking](#)

[ConcurrentQueues]: [p0260r1: Concurrent Queues](#)

[AsyncParallel]: [p0361r1: Invoking Algorithms Asynchronously](#)

[Latches-and-Barriers]: [p0666r1: Revised Latches and Barriers for C++20](#)

[ConcurrencyTS]: [P0159: Concurrency TS1](#)

[RCU-HazardPtr]: [p0566r4: Concurrent Data Structures Hazard Pointer and Read-Copy-Update \(RCU\)](#)

[Networking-and-Threadpools]: [p0399r0: Networking TS & Threadpools](#)

[CoroAlloc]: [Godbolt: Coroutines using allocator example](#)

[Range-v3]: [Range-v3 on Github](#)

[RangesTS]: [N4685: C++ Extensions for Ranges](#)

[UFC]: [N4474: Unified Call Syntax](#)

[RxMarbles]: [Asynchronous stream composition](#)

[RxCpp]: [Reactive Extensions for C++](#)

[P0701R2]: [p0701r2: Back to the std2::future Part II](#)

[P0904R0]: [P0904r0: A strawman Future API](#)

[P0676R0]: [P0676r0: Towards a Good Future](#)

[P0797R1]: [P0797R1: Handling Concurrent Exceptions with Executors](#)

10. Appendix

10.1. Full listing of hand-crafted state machine matching the example in "Networking Technical Specification" section of the paper.

Compare the following short fragment:

```
task<> session(tcp::socket s, size_t block_size) {
    std::vector<char> buf(block_size);
    for (;;) {
        size_t n = co_await async_read_some(s, buffer(buf.data(), block_size));
        co_await async_write(s, buffer(buf.data(), n));
    }
}
```



```

task<> server(io_context& io, tcp::endpoint const& endpoint, size_t block_size)
    tcp::acceptor acceptor(io, endpoint);
    acceptor.listen();
    for (;;)
        spawn(io, session(co_await async_accept(acceptor), block_size));
}

```

and the equivalent code without coroutines:

```

struct session {
    session(io_context& io, ip::tcp::socket s, size_t block_size)
        : io_context_(io), socket_(std::move(s)), block_size_(block_size), buf_(bl
          read_data_length_(0) {}

    void start() {
        std::error_code set_option_err;
        if (!set_option_err) {
            socket_.async_read_some(buffer(buf_.data(), block_size_),
                                     [this](auto ec, auto n) { handle_read(ec, n); });
            return;
        }
        net::post(io_context_, [this] { destroy(this); });
    }

    void handle_read(const std::error_code& err, size_t length) {
        if (!err) {
            read_data_length_ = length;
            async_write(socket_, buffer(buf_.data(), read_data_length_),
                        [this](auto ec, auto) { handle_write(ec); });
            return;
        }
        net::post(io_context_, [this] { destroy(this); });
    }

    void handle_write(const std::error_code& err) {
        if (!err) {
            socket_.async_read_some(buffer(buf_.data(), block_size_),
                                     [this](auto ec, auto n) { handle_read(ec, n); });
            return;
        }
        net::post(io_context_, [this] { destroy(this); });
    }

    static void destroy(session* s) { delete s; }

private:
    net::io_context& io_context_;
    net::ip::tcp::socket socket_;

```

```

    size_t block_size_;
    std::vector<char> buf_;
    size_t read_data_length_;
};

struct server {
    server(io_context& io, const ip::tcp::endpoint& endpoint, size_t block_size)
        : io_context_(io), acceptor_(io, endpoint), block_size_(block_size) {
        acceptor_.listen();
        start_accept();
    }

    void start_accept() {
        acceptor_.async_accept([this](auto ec, auto s) { handle_accept(ec, std::move
    }

    void handle_accept(std::error_code err, ip::tcp::socket s) {
        if (!err) {
            session* new_session = new session(io_context_, std::move(s), block_size_)
            new_session->start();
        }
        start_accept();
    }

private:
    io_context& io_context_;
    ip::tcp::acceptor acceptor_;
    size_t block_size_;
};

```